Torneo de Programación de Comportamientos 3: Instrucciones

Google Developer Group
Universidad de Valladolid

Daniel López Martínez Pablo Cabrera Vara

Índice

1. Introducción **1**

2. Entorno y Mecánicas **1**
2.1. Personaje: comportamiento y acciones 2
2.2. Pez globo: comportamientos y acciones 2
2.3. Sobre la muerte súbita 2
2.4. Sistema de colisiones 3
2.5. Sistema de creación de mapas 4

3. Puesta en marcha y *setup* inicial **5**

4. Guía de Funciones Disponibles **6**
4.1. Acciones del personaje 6
4.2. Información del entorno 7

1. Introducción

Bienvenidos al tercer torneo de programación de comportamientos, en el que definiréis el algoritmo que deberá seguir vuestro personaje para adaptarse a todas las estrategias de vuestro contrincante. Como bien dice el nombre, en este torneo deberéis programar en código Python el comportamiento de vuestro personaje. Para ello tendréis acceso a un montón de funciones que os permitirán obtener información del entorno y de vuestro rival en cada momento de la partida, junto con funciones que activarán distintas acciones en vuestro personaje. Pensad detenidamente en cómo actuará vuestro rival y preparad contramedidas para cada acción que pueda tomar para obtener la victoria bajo cualquier posibilidad.

2. Entorno y Mecánicas

El juego sobre el que se desarrolla el torneo es un género de plataformas con muerte súbita activable por tiempo. La meta es hacer que el personaje rival sea eliminado antes que el tuyo, ya sea por causas del entorno (peces globo mortales) o por la muerte súbita.
El entorno donde se desarrolla la acción es una zona cuadriculada cerrada, tipo "cueva", con plataformas flotantes a las que se podrá acceder saltando. Pero cuidado, en esta "cueva" habitan peces globo que pueden convertirse en armas letales en las manos equivocadas.

2.1. Personaje: comportamiento y acciones

Los personajes controlables se podrán desplazar horizontalmente (en el eje x) y verticalmente (en el eje y) mediante saltos. Por lo tanto, las acciones que podrán realizar son:

- Movimiento horizonte básico, es decir, en el eje x.
- Movimiento vertical, mediante saltos normales que tienen una altura de 2 bloques del escenario, o saltos dobles o "cohete", utilizando un pez globo (véase sección sobre pez globo).
- Agarrar pez globo.
- Lanzar pez globo en una dirección determinada.

La velocidad a la que se mueve un personaje es constante y es de 2 píxeles por frame.

2.2. Pez globo: comportamientos y acciones

Los peces globo son criaturas no controlables directamente que se encuentran en estado *no dañino* al comienzo de la partida. En este estado estarán flotando en una única posición, la cual no tiene por qué estar sustentada por un suelo físico. En el estado *no dañino* del pez globo, un personaje jugable puede interactuar con el pez de dos maneras:

- Acercarse y agarrar el pez globo para llevarlo en la mano.
- Acercarse y patearlo por simple contacto con el pez globo, haciendo que entre en *movimiento* en la dirección del movimiento del personaje.

Cuando un personaje jugable tiene el pez globo agarrado, puede realizar varias acciones con él:

- Lanzar el pez globo en una dirección específica para que entre en *movimiento*.
- Realizar un "salto cohete" o doble salto cuando se está en el aire, lanzando de esa manera el pez globo hacia abajo. Cuando el pez globo toque el suelo, pasará a estado *no dañino*, quedándose quieto en el sitio.
- Mantener el pez en la mano hasta nuevo aviso.

Cuando el pez globo se encuentra en *movimiento*, se convierte en un objeto imparable que arrasa con cualquier personaje que se interponga en su camino, por lo que, si no quieres perder la partida, procura esquivarlo saltando o evitando su trayectoria. Su movimiento es únicamente horizontal, rebotando en la primera pared que encuentre en su camino y a una velocidad de 4 píxeles por frame. Esta velocidad es mayor a la del personaje, por lo que ten cuidado si lo pateas muy cerca de una pared, ya que podrías no tener tiempo para reaccionar.

2.3. Sobre la muerte súbita

La muerte súbita es un evento de la partida que ocurre cuando han pasado 7 segundos de partida. En este evento, los muros laterales se cerrarán sobre los personajes poco a poco, a una velocidad de 1 píxel por frame. Si uno de los muros laterales toca a un personaje, este perderá la partida automáticamente. Por lo tanto, dominar el espacio central será imperativo si la partida se alarga.

2.4. Sistema de colisiones

Las colisiones funcionan tomando todo objeto del escenario como un rectángulo y viendo si entre ellos hay intersección. Para mejorar la explicación de las colisiones, se presentan en esta sección una serie de imágenes con distintas marcas. El punto rojo intenso indica el origen de coordenadas del objeto y el punto de coordenadas que nos devolverán las funciones acerca del mismo. El recuadro rojo semitransparente corresponde al área de colisión del objeto.

Como podemos ver en la Figura 1, los personajes y los bloques (tiles) que componen el escenario son de 32x32 píxeles.

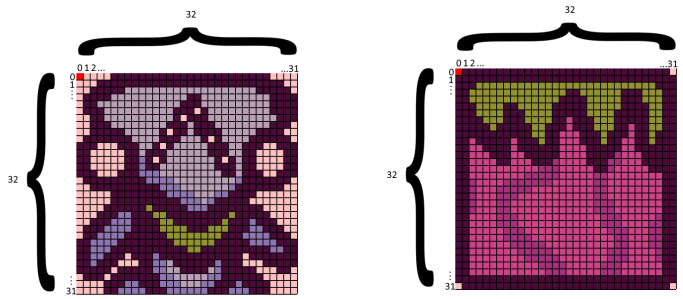

Figura 1: Colisiones de los jugadores y tiles del escenario

En la Figura 2 se observa que el pez globo, cuando es pequeño, es de 16x32 píxeles y su origen de coordenadas está fuera de su colisión 8 píxeles a la izquierda, en la esquina superior izquierda. Cuando el pez está inflado, su colisión pasa a ser de 28x25 píxeles y su origen está 2 píxeles fuera de su colisión en el eje X y 3 en el eje Y.

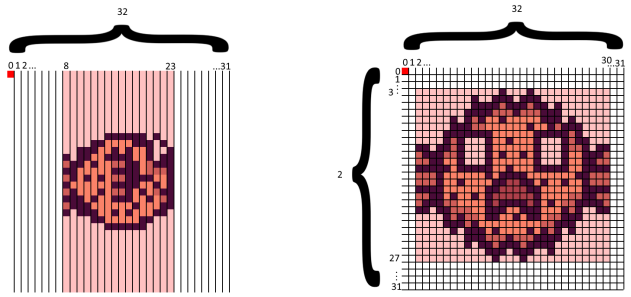

Figura 2: Colisiones de los peces

El área de colisión de los muros de la muerte súbita, que se representa en la Figura 3, consiste en una sola columna de píxeles que ocupan la altura completa del mapa. En el caso del muro izquierdo, su área de colisión tiene origen de coordenadas en la esquina superior derecha del dibujo del muro, mientras que en el muro derecho, su área de colisión tiene origen de coordenadas en la esquina superior izquierda. La dirección de este origen de coordenadas en función del mapa total es la que se devuelve en las funciones de posición de la muerte súbita. Por lo tanto, si el muro izquierdo tiene una posición en el eje X mayor

que la del personaje jugable, este perderá; y si el muro derecho tiene una posición en el eje X menor que la del personaje jugable + 32 (el ancho de nuestro personaje), este perderá.

Figura 3: Colisiones de los pinchos de la muerte súbita

2.5. Sistema de creación de mapas

El TPC3 cuenta con un sistema de creación de niveles hecho para que los participantes desarrollen su creatividad y prueben cómo actúa su comportamiento programado en distintas situaciones.
El proyecto cuenta con una carpeta llamada `levels` en la cual cualquier archivo llamado `level_i.txt` siendo i el número de orden de ejecución del nivel, será procesado y jugado por los bots. Este índice empieza en 0 y parará cuando no haya ningún nivel con el índice actual.
Los niveles funcionan como matrices de caracteres y usan los siguientes elementos; nótese que es importante respetar las minúsculas y mayúsculas:

- 'o' es usado para determinar que en esa casilla hay un bloque sólido del escenario (tierra).
- 'A' para determinar la posición inicial del jugador 1.
- 'B' para la posición inicial del jugador 2.
- 'F' para la posición inicial de un pez. El juego puede tener tantos peces como el usuario quiera.
- ' ' o cualquier otro carácter es tomado como aire.

Se añadirá un recuadro o borde de bloques sólidos al nivel que hemos creado para evitar que algún objeto salga de este. Además, si nuestro nivel no es rectangular, se añadirá aire en los espacios vacíos para completar la matriz, de tal manera que todos los mapas tendrán siempre una forma rectangular. En la Figura 4 se pueden observar ejemplos de escenarios y con su formato en texto plano.

Figura 4: Ejemplos de niveles

3. Puesta en marcha y *setup* inicial

Una vez que hayamos descargado y descomprimido el proyecto, nos encontramos con 4 carpetas y un archivo llamado `main.py`. En la carpeta `assets` se encuentran todas las imágenes del proyecto y en la carpeta `src` la lógica del mismo; estas dos carpetas no deben ser modificadas y deben ser ignoradas por el usuario. En la carpeta `levels` como se ha descrito anteriormente, se encuentran los niveles que el juego procesará y el usuario podrá añadir tantos como quiera. En `submissions` es donde el usuario creará su bot. Encontrará en esta carpeta un fichero de ejemplo, uno que explica brevemente las funciones en funcionamiento, uno para jugar con el teclado, ideal para jugar contra el bot programado para comprobar su desempeño, y uno llamado `pon_tu_nombre` al que el usuario tiene que cambiar el nombre poniendo el de su equipo.

```python
from src.controller import Controller

class Submission(Controller):
    def info(self):
        self.team_name = "Nombre de ejemplo"
        self.look = 5 # Pon un número del 1 al 5
        self.color = 3 # Pon un número del 0 al 3

    def behavior(self):
        self.go_left()
        self.jump()
```

Figura 5: Código a modificar

En la Figura 5 se muestra un ejemplo de código que tendréis que entregar. Este se divide en dos funciones:

- `info(self)`: en esta podemos definir nuestras variables que queramos conservar durante toda la partida. Ya cuenta con dos variables que customizan a nuestro personaje y una que da nombre a nuestro equipo; no te olvides de editarlas.
- `behavior(self)`: esta función se ejecutará cada frame del juego, es decir, 60 veces por segundo, ya que el juego se ejecuta a 60 frames por segundo. En el escribiremos los botones que queremos pulsar y decidiremos cuándo pulsarlos en función de las funciones que se explican más adelante.

Para ejecutar el proyecto, nos pondremos en la carpeta raíz y ejecutaremos `python main.py bot_a bot_b` poniendo como bot_a y bot_b los nombres de los archivos de los bots a probar sin poner el .py. Para ejecutar este proyecto necesitamos python3 y tener instalada la librería pygame. Si no la tenemos, la podemos instalar haciendo *pip install pygame*.

4. Guía de Funciones Disponibles

En esta guía se definen las distintas funciones de Python que se pueden utilizar para obtener información del entorno y que vuestro personaje jugable pueda realizar acciones sobre el mismo. Todas estas funciones heredan de la clase `Controller` y requieren de una serie de parámetros de entrada y devolverán distintos valores como salida. Las funciones a las que llaméis dentro de la función `behavior()` se ejecutarán en cada frame del juego una única vez, teniendo en cuenta que el juego se ejecuta a 60 frames por segundo. Es importante tener en cuenta que todas las funciones deben ejecutarse con `self.<nombreDeLaFuncion>`, si se omite el `self.` no funcionarán.

4.1. Acciones del personaje

Las funciones de Python disponibles para realizar acciones con vuestro personaje son las siguientes:

- `go_right()`: función para moverse hacia la derecha a velocidad de 2 píxeles por frame.
- `go_left()`: función para moverse hacia la izquierda a velocidad de 2 píxeles por frame.
- `jump()`: función para saltar dos bloques de altura.
- `grab()`: función para agarrar un pez globo si se entra en contacto con el pez globo durante el frame de ejecución.
- `throw_right()`: lanza un pez globo hacia la derecha si tenía uno ya agarrado.
- `throw_left()`: lanza un pez globo hacia la izquierda si tenía uno ya agarrado.

- `throw_down()`: lanza un pez globo hacia abajo si tenía uno ya agarrado y se encuentra en el aire, propulsando al personaje jugable hacia arriba igual que en un salto normal ("salto cohete"). Cuando el pez globo toque el suelo pasa a estado *no dañino*.

4.2. Información del entorno

La mayoría de las funciones para obtener información del entorno utilizan un sistema de coordenadas X e Y en función del número de píxeles del mapa de juego. Este sistema de coordenadas cuenta la esquina superior izquierda como la dirección `(0,0)` y su valor crece conforme avanzas hacia la derecha y hacia abajo en cada eje correspondiente; por lo tanto, **no hay coordenadas negativas**. Tanto los personajes como los bloques de terreno ocupan un espacio de 32x32 píxeles, es decir, 32 píxeles en el eje X y 32 en el eje Y. Sin embargo, los peces globo en estado en *movimiento* ocupan un espacio de 28x25, y en estado *no dañino* ocupan un espacio menor, de 16x32 píxeles.

Todas las funciones que devuelven coordenadas de un objeto en el mapa las devuelven en función de la esquina superior izquierda del objeto en cuestión; por lo tanto, si un personaje está en la esquina superior izquierda del mapa, su posición en coordenadas sería `(0,0)`. Esto funciona así, exceptuando la muerte súbita, cuyo comportamiento se explica en la Sección 2.4.

Las funciones de Python disponibles para obtener información del entorno en cualquier momento son las siguientes:

- `get_x()`: función que devuelve la posición en el eje X, en píxeles del mapa, de tu personaje jugable.
- `get_y()`: función que devuelve la posición en el eje Y, en píxeles del mapa, de tu personaje jugable.
- `get_y_speed()`: función que devuelve la velocidad de caída de tu personaje jugable, en píxeles por frame, teniendo en cuenta que en el eje Y sí hay aceleración por caída.
- `get_enemy_x()`: función que devuelve la posición en el eje X, en píxeles del mapa, de tu enemigo.
- `get_enemy_y()`: función que devuelve la posición en el eje Y, en píxeles del mapa, de tu enemigo.
- `is_pixel_ground(x, y)`: función que, a partir de unas coordenadas X e Y, en píxeles del mapa, te dice si dichas coordenadas corresponden con terreno pisable o, por el contrario, es aire.
- `get_level_x_pixel_size()`: función que devuelve el número de píxeles totales del mapa en el eje X, es decir, el ancho del mapa.
- `get_level_y_pixel_size()`: función que devuelve el número de píxeles totales del mapa en el eje Y, es decir, la altura del mapa.
- `is_sudden_death_active()`: función que devuelve un booleano que indica si la muerte súbita está activa o no.
- `get_left_sudden_death()`: función que devuelve la posición en el eje X del muro izquierdo de muerte súbita.
- `get_right_sudden_death()`: función que devuelve la posición en el eje X del muro derecho de muerte súbita.

Otras funciones más complejas son:

- `get_list_fish_pos()`: función que devuelve una lista de Python con las posiciones de los peces globo, de tal manera que en la primera dimensión o `[]` tenemos el número de pez globo, y en la segunda dimensión la posición en el eje X en el índice 0, y la posición en el eje Y en el índice 1.

Ejemplos:
`self.get_list_fish_pos()[0][0]` devuelve la posición del pez globo 0 en el eje X.
`self.get_list_fish_pos()[1][1]` devuelve la posición del pez globo 1 en el eje Y.

- `get_list_fish_state()`: función que devuelve una lista de enteros que representan el estado actual de cada pez globo. Siendo 0 que el pez está en estado *no dañino*, 1 que el pez está moviéndose hacia la derecha, 2 que el pez está moviéndose hacia la izquierda y 3 que el pez está en caída libre hasta que toque el suelo. Si el pez está en caída libre, cuando toque el suelo se dirigirá hacia la última dirección registrada.

- `is_grabbing_fish()`: función que devuelve un booleano que indica si tu personaje jugable está agarrando un pez globo actualmente; por lo tanto, devuelve *True* si está agarrando un pez globo y *False* si no.

- `is_enemy_grabbing_fish()`: función que devuelve un booleano que indica si tu enemigo está agarrando un pez globo actualmente; por lo tanto, devuelve *True* si está agarrando un pez globo y *False* si no.

- `get_tile_level_matrix()`: función que devuelve el texto plano del estado del mapa en tiempo real, en el formato explicado en la sección 2.5, añadiendo también los bordes del mapa, en bloques de 32x32 píxeles. Este mapa mostrará la posición de los tú personaje jugable con una 'A' y el del enemigo con una 'B', y la posición de los peces con el número correspondiente a su estado actual: '0' estado *no dañino*, '1' moviéndose a la derecha, '2' moviéndose a la izquierda y '3' en caída libre. Cada elemento de la lista de listas es un carácter. Dado que la función devuelve una lista de listas, se accede a los ejes de manera invertida, es decir, `mapa[eje_y][eje_x]`.

- `get_pixel_level_matrix()`: función que devuelve el estado del mapa en tiempo real, en el mismo formato que la función `get_tile_level_matrix()`, pero representado en píxeles en vez de bloques de 32x32. Esto significa que la posición de los distintos elementos del escenarios se representará como un bloque de caracteres idénticos, atendiendo a los píxeles que ocupa su área de colisión, explicadas en la sección 2.4. Por ejemplo, si tu personaje jugable está en la posición x = 90, y = 65; la lista que devuelve esta función marcará con 'A' todos los píxeles desde x = 90, y = 65 hasta x = 90 + 31, y = 65 + 31. Dado que la función devuelve una lista de listas, se accede a los ejes de manera invertida, es decir, `mapa[eje_y][eje_x]`.